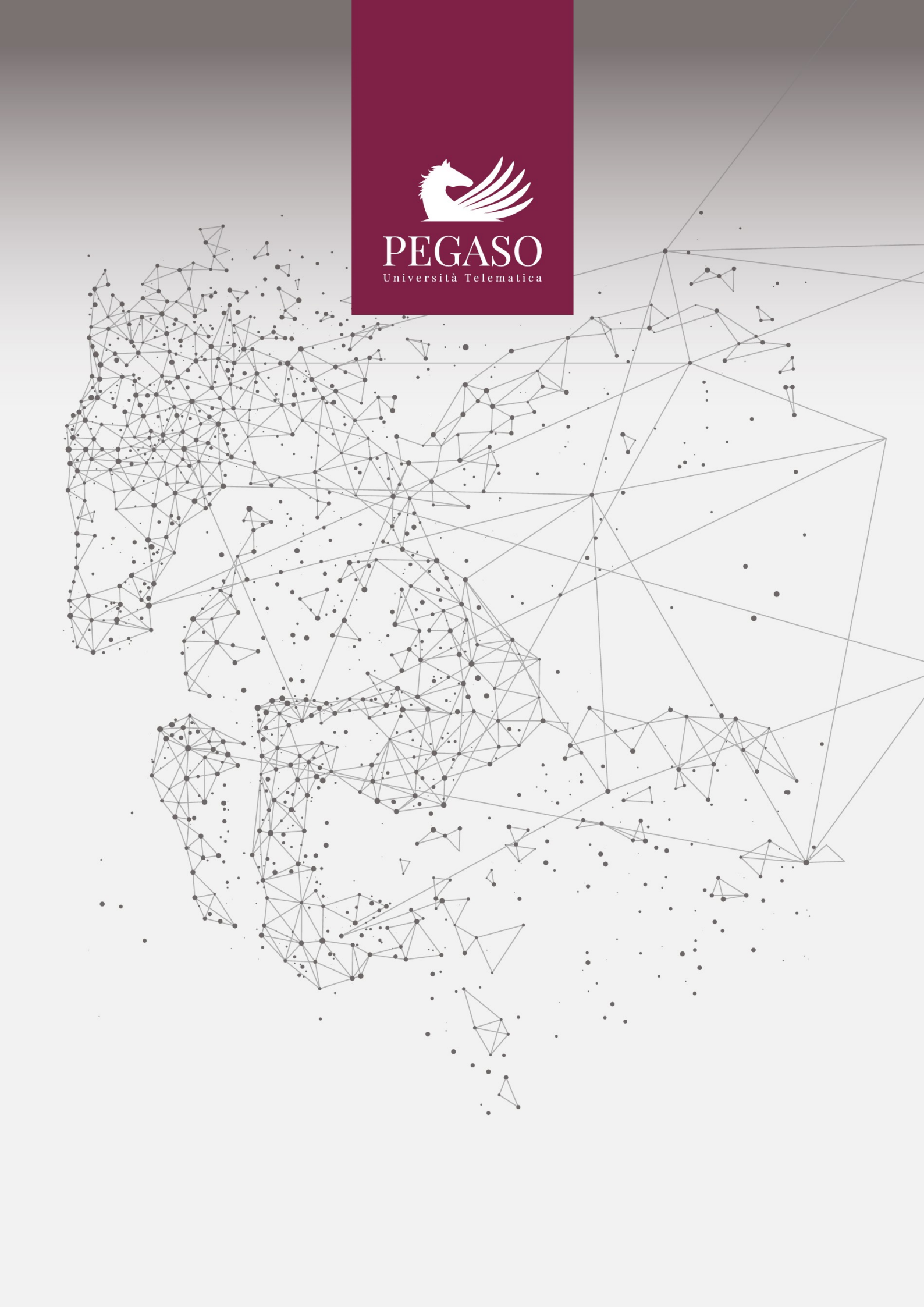

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INTRODUZIONE ALLE FUNZIONI 4
3. PASSAGGIO DI PARAMETRI ALLE FUNZIONI 8
4. SCRIVERE UN PROGRAMMA CHE USA LE FUNZIONI 10
5. REGOLE DI VISIBILITÀ DELLE FUNZIONI 13
6. FUNZIONI DELLE LIBRERIE 16
BIBLIOGRAFIA 18

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono le principali caratteristiche delle funzioni nel linguaggio di programmazione Python, incluso il passaggio di parametri e la visibilità delle variabili. Verrà illustrato, inoltre, un esempio di approccio top-down per la progettazione di un programma composto da funzioni.

Infine, verrà presentata la standard library di Python e introdotto il modulo matematico math.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione alle funzioni

Una **funzione** è un gruppo di istruzioni all'interno di un programma che svolgono un compito specifico. Invece di scrivere un programma molto grande come un'unica sequenza di istruzioni, è, infatti, possibile scrivere funzioni più piccole ciascuna delle quali svolge una funzionalità specifica. Una funzione è caratterizzata da:

1. nome;
2. eventuali parametri;
3. eventuale valore restituito.

I vantaggi della modularizzazione di un programma con le funzioni sono molteplici:

- codice più semplice;
- **riuso del codice**: si scrive il codice una volta e si utilizza diverse volte;
- test e debugging più semplice (a livello di funzione);
- sviluppo più semplice;
- lavoro in team semplificato poiché ogni membro del team può scrivere una funzione diversa.

Il formato generale della **definizione di una funzione** in Python riportato di seguito è composto da due parti:

```
def nome_funzione():
    istruzione
    istruzione
```

1. **header della funzione**, la prima linea della funzione che include la keyword **def** e il nome della funzione seguito da parentesi tonde e dai due punti. I nomi delle funzioni, per i quali solitamente si utilizza un **verbo**, devono essere significativi rispetto allo scopo della funzione;
2. **blocco**, cioè l'insieme di istruzioni che caratterizzano la funzione.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le funzioni sono dei **nomi simbolici** che devono seguire alcune regole di denominazione:

- non è possibile utilizzare le key word come nomi di funzioni;
- non possono contenere spazi;
- il primo carattere deve essere una lettera o un underscore;
- gli altri caratteri possono essere lettere, numeri e underscore;
- i nomi delle funzioni sono case-sensitive.

Volendo operare una semplice classificazione delle funzioni, è possibile affermare che esistono principalmente due tipi di funzioni **in base al valore restituito**:

1. **funzioni void**, eseguono semplicemente le istruzioni che contengono e poi terminano (non viene restituito nessun valore);
2. **funzioni value-returning**, eseguono le istruzioni contenute e restituiscono un valore all'istruzione che l'ha chiamata.

   Le funzioni input(), int(), e float() sono esempi di funzioni value-returning.

Un esempio di dichiarazione di funzione void potrebbe essere quello corrispondente al seguente codice sorgente in cui si richiede di visualizzare a schermo alcune informazioni relative ad un indirizzo di fantasia:

```
def stampa_dati():
    print("Indirizzo:")
    print("Via Roma, 1")
    print("00184 Milano")
```

Un esempio di dichiarazione di funzione value-returning, invece, potrebbe essere quello corrispondente al seguente codice sorgente in cui si richiede di calcolare e restituire la somma di due valori passati come parametri alla funzione **somma**( ):

```
def somma(a,b):
    return a+b
```

Per poter eseguire una funzione che è stata già definita è necessario chiamarla. Quando una funzione viene chiamata da un'altra funzione detta **invocante** (o **chiamante**), l'interprete salta alla funzione ed esegue le istruzioni nel blocco. Una volta terminate le istruzioni nella funzione,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

l'interprete salta indietro alla parte del programma (funzione invocante) che ha chiamato la funzione (**function return**).

In Python, esiste una funzione principale, la **funzione main**, che viene definita e chiamata quando il programma viene avviato. Quest'ultima, poi, chiama le altre funzioni quando sono necessarie e definisce la logica principale del programma.

Si riporta un programma che cerca di comprendere tutti gli aspetti visti finora: funzioni void, value-returning, main e loro chiamata. Si noti la chiamata alla funzione main() al termine del codice sorgente che darà avvio all'intero programma e la definizione di tre funzioni. In particolare, la funzione main richiamerà le funzioni stampa_dati() e somma(a,b):

```
def stampa_dati():
    print("Indirizzo:")
    print("Via Roma, 1")
    print("00184 Milano")


def somma(a,b):
    return a+b


def main():
    stampa_dati()
    print(somma(3,2))


main()
```

Il risultato dell'elaborazione sarà il seguente:

```
Indirizzo:
Via Roma, 1
00184 Milano
5
```

Si forniscono ora una serie di accorgimenti importanti nella scrittura delle funzioni:

- ciascun blocco deve essere correttamente indentato. Le istruzioni nei blocchi devono cominciare con lo stesso numero di spazi bianchi;
- per l'indentazione si utilizzano i tab e gli spazi, ma non si possono usare entrambi altrimenti l'interprete Python non funziona correttamente (nel corso utilizzeremo i tab);

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- l'IDLE di Python indenta automaticamente le linee in un blocco. A tal proposito, occorre prestare molta attenzione perché, a volte, a causa di un'indentazione errata è possibile che il programma non si comporti come desiderato;
- le linee bianche all'interno del codice sorgente vengono ignorate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Passaggio di parametri alle funzioni

Spesso risulta utile non solo chiamare una funzione, ma passarle uno più dati. I dati inviati ad una funzione sono chiamati **argomenti della funzione**. Se si vogliono passare degli argomenti ad una funzione in fase di chiamata della stessa, bisogna fornire alla funzione uno o più parametri. In Figura 1, si riporta, a titolo di esempio, il passaggio di un parametro in fase di chiamata alla funzione **raddoppia**( ).



Figura 1. Passaggio di un parametro alla funzione raddoppia ()

In Figura 2, si riporta, a titolo di esempio, il passaggio di due parametri in fase di chiamata alla funzione **moltiplica**( ).

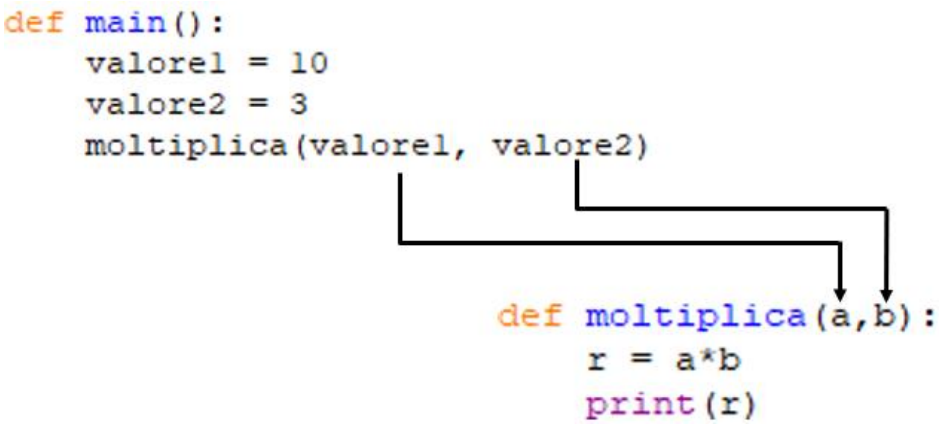


Figura 2. Passaggio di due parametri alla funzione moltiplica ()

In Python, le variazioni che vengono apportate ai parametri all'interno delle funzioni non apportano modifiche agli argomenti nella funzione chiamante (o funzione invocante). Questo

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

meccanismo è noto come **passaggio di parametri per valore**. Per dimostrare la mancanza di modifiche apportate nella funzione chiamata, si riporta un semplice codice sorgente Python. In particolare, la funzione *main* chiama la funzione *azzera* passandole il valore 10. La funzione azzera porrà il valore ricevuto a zero e lo visualizzerà a schermo. Quando l'interprete tornerà sulla funzione main, stamperà il valore della variabile *val* che sarà sempre uguale a 10 poiché le modifiche apportate dalla funzione *azzera*( ) non si rifletteranno nella funzione *main*( ).

```
def azzera(val):
    val=0
    print("La variabile val nella funzione azzera è uguale a:", val)


def main():
    val = 10
    azzera(val)
    print("La variabile val nel main è uguale a:", val)

main()
```

Il risultato dell'elaborazione del codice sarà:

```
La variabile val nella funzione azzera è uguale a: 0
La variabile val nel main è uguale a: 10
```

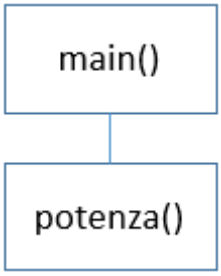
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

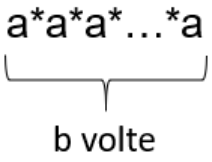
4. Scrivere un programma che usa le funzioni

Solitamente, nello sviluppo di un programma con le funzioni, si affronta il problema da risolvere con un approccio **top-down**. Un problema, infatti, può essere scomposto in sottoproblemi che verranno poi risolti con l'uso di una funzione. Immaginiamo di dover risolvere il problema del calcolo della potenza, ma di non avere a disposizione l'operatore matematico necessario di elevazione a potenza. L'unico operatore matematico che potrà essere utilizzato è quello dell'**operatore + di somma**, anche se in prima istanza possiamo utilizzare anche l'**operatore * di moltiplicazione**.

Uno primo schema per l'approccio top-down di scomposizione del problema potrebbe essere il seguente (avendo a disposizione l'operatore di moltiplicazione):



Prima di procedere, analizziamo il significato di potenza di un numero: a^b significa che *a* deve essere moltiplicato per sé stesso *b* volte:



Nella funzione potenza() potremo, quindi, utilizzare un ciclo for che effettua *b* volte la moltiplicazione di *a* (si noti che la variabile locale *c* viene inizializzata ad 1 perché 1 è l'elemento neutro per la moltiplicazione e permette, quindi, di calcolare prodotti successivi):

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def potenza(a,b):
    c=1
    for i in range(b):
        c=c*a
    return c

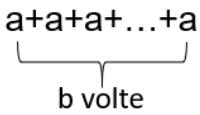

def main():
    a= int(input("Inserire la base: "))
    b= int(input("Inserire l'esponente: "))
    print(potenza(a,b))


main()
```

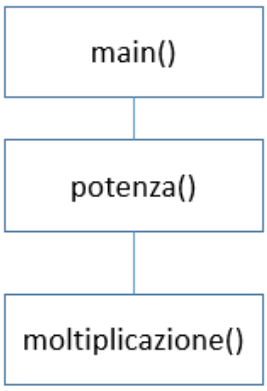
Il risultato dell'elaborazione per una base pari a 2 e un'esponente pari a 3 è il seguente:

```
Inserire la base: 2
Inserire l'esponente: 3
8
```

Ora, ipotizzando di non poter più utilizzare l'operatore moltiplicazione, ma solo quello di somma, ragioniamo sul significato della moltiplicazione. La moltiplicazione *a*b* significa che *a* deve essere sommato a sé stesso *b* volte:



Nella funzione moltiplicazione(), quindi, potremo utilizzare un ciclo for che effettua *b* volte la somma di *a*. Il nuovo schema per l'approccio top-down di scomposizione del problema di elevazione a potenza potrebbe essere il seguente (avendo a disposizione solo l'operatore di addizione):



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice complessivo scritto in Python che risolve il problema è:

```
def moltiplicazione(a,b):
    c=0
    for i in range(b):
        c=c+a
    return c


def potenza(a,b):
    c=1
    for i in range(b):
        c= moltiplicazione(c,a)
    return c



def main():
    a= int(input("Inserire la base: "))
    b= int(input("Inserire l'esponente: "))
    print(potenza(a,b))


main()
```

che produce in output:

```
Inserire la base: 2
Inserire l'esponente: 3
8
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Regole di visibilità delle funzioni

In Python, così come altri linguaggi di programmazione, esistono due tipi di variabili: **locali** e **globali**.

Le **variabili locali** sono variabili a cui viene assegnato un valore all'interno di una funzione.

Tali variabili appartengono alla funzione che le ha create e solo le istruzioni della stessa funzione possono accedervi. Se un'altra funzione cerca di accedervi, verrà riscontrato un errore.

In merito allo **scope** di una variabile, ovvero la sua visibilità (la parte del programma che può accedere alla variabile), le variabili locali sono visibili solo nella funzione in cui vengono create.

Ciascuna funzione "vede", infatti, le proprie variabili locali. A titolo di esempio, il codice sorgente che segue non rispetta le regole di visibilità delle variabili locali in quanto la variabile *a* è una variabile locale al main e la funzione stampa() non ne ha visibilità.

```
def stampa():
    print(a)

def main():
    a=3
    stampa()

main()
```

Il codice seguente, invece, mostra un'applicazione delle regole di visibilità di una variabile locale. La variabile locale *a* nella funzione stampa è diversa dalla variabile locale *a* della funzione main, come schematizzato in Figura 3.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def stampa():
    a=7
    print(a)

def main():
    a=3
    stampa()

main()
```

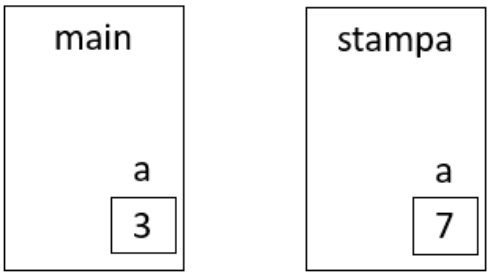


Figura 3. Schema dell'utilizzo di due variabili locali aventi lo stesso nome in due funzioni diverse

Le **variabili globali**, invece, sono create da un'istruzione di assegnazione fuori dalle dichiarazioni delle funzioni. Lo **scope** (visibilità) delle variabili globali, al contrario di quelle locali, è tutto il programma inclusi i blocchi delle funzioni. Per poter cambiare il valore di una variabile globale in una funzione del programma, è necessario dichiarare nuovamente la variabile utilizzando **keyword global**. Nel codice seguente, *my_value* è una variabile globale visibile dalla funzione *show_value*:

```
# Crea una variabile globale.
my_value = 10

# La funzione show_value stampa
# il valore della variabile globale.
def show_value():
    print(my_value)

# Chiama la funzione show_value.
show_value()
```

Nel codice seguente, con l'utilizzo della keyword global, può essere cambiato il valore della variabile globale number all'interno di una o più funzioni.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
# Crea una variabile globale.
number = 0

def main():
    global number
    number = int(input('Inserisci un numero: '))
    show_number()

def show_number():
    print('Il numero che hai inserito è', number)

# Chiama la funzione main.
main()
```

Nella programmazione è consuetudine ridurre al minimo l'utilizzo delle variabili globali privilegiando il ricorso al passaggio di parametri alle funzioni. In questo modo, infatti, le funzioni saranno indipendenti da un particolare programma (e dalle sue variabili) e potranno essere utilizzate anche in altri programmi (**riuso del codice**). Le variabili globali rendono, inoltre, il debugging più complesso. In aggiunta, se le variabili globali sono richiamate in diversi punti del codice possono causare il valore di una o più variabili sbagliato. Infine, le variabili globali rendono un programma difficile da capire.

Discorso diverso è quello relativo alle **costanti globali**, cioè variabili non modificabili che non prevedono controindicazioni al loro uso. È possibile simulare costanti globali in Python dichiarandole come variabili globali e non dichiarandole nuovamente nelle funzioni con la keyword global. Solitamente, per convenzione, per le costanti globali si utilizzano nomi scritti tutti in maiuscolo (es. PI_GRECO = 3.14). Nessuna funzione dovrà utilizzare la keyword global nella dichiarazione della funzione stessa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Funzioni delle librerie

La **standard library** è una libreria di funzioni già scritte che sono contenute direttamente in Python. La standard library è composta da più **moduli**, a loro volta costituiti da funzioni che permettono di eseguire delle attività necessarie per il programmatore. Esempi di funzioni della standard library sono print( ), input( ) e range( ).

I programmatori vedono tali funzioni come delle "black box" (scatole nere) in cui si conosce input e output, ma non l'algoritmo di elaborazione che dato l'input permette di calcolare l'output. Alcune funzioni di libreria sono incluse nell'interprete Python (basta chiamarle).

I **moduli** sono dei file che contengono le funzioni della standard library e aiutano ad organizzare le funzioni della libreria non incluse nell'interprete. Per chiamare una funzione inclusa in un modulo, si utilizza l'**istruzione import** (tale istruzione si riporta in cima al codice sorgente del programma). Il formato dell'importazione di un modulo è il seguente:

```
import nome_modulo
```

Un modulo molto utilizzato in Python è il **modulo matematico *math*** utile per funzioni trigonometriche, esponenziali, ecc. Tutte le funzioni del modulo math di Python disponibili all'indirizzo: https://docs.python.org/3/library/math.html

Nel codice seguente, si importa il modulo math e si calcola il valore della funzione seno per pi greco radianti (180°):

```
import math

PI_GRECO = 3.14


def main():
    a = math.sin(PI_GRECO)
    print(a)

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per approfondimenti sulla standard library, il sito ufficiale è disponibile al seguente link: https://docs.python.org/3/library/.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).